



EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

569406 PROGRAMMING LANGUAGES

2024–2025 SPRING SEMESTER

PROJECT-1 REPORT

(Lexical Analyzer for the Plus++ Language)

DELIVERY DATE

30/05/2025

PREPARED BY

05230000350, Ceylin TOR

05220001067, Aykut HACIOĞLU

Contents

1) PROJECT DEFINITION AND EXPLANATION	2
2) CODE DESCRIPTION	4
2.a) Libraries & Constants	4
2.b) Symbol Table	4
2.c) Token Classification Functions	5
2.d) Error Reporting.....	6
2.e) Main Analysis Function	6
2.f) Entry Point.....	12
3) ERROR HANDLING	12
3.a) Invalid or Incorrect Identifiers.....	12
3.b) Unclosed Comment.....	13
3.c) Unclosed String	14
3.d) Invalid Numbers	15
3.e) Missing "{" after repeat block	16
3.f) More "}"	16
4) GIVEN SCRIPT (NO ERROR)	17

1) PROJECT DEFINITION AND EXPLANATION

This project implements a **Lexical Analyzer** for a simple educational programming language named **Plus++**, using the C programming language. The analyzer reads source code files with the .plus extension and generates a corresponding .lxoutput file listing all valid lexical tokens.

Purpose

The purpose of this project is to:

- Extract meaningful tokens (keywords, identifiers, literals, operators, etc.) from a source program.
- Validate language-specific rules for identifiers, constants, and structures.
- Detect and report common lexical errors such as undeclared variables, invalid characters, and unclosed strings or comments.
- Maintain a **symbol table** to track declared variables and enforce scope rules.

Functionality

The analyzer performs the following key tasks:

- **File Reading:** Reads .plus source files line by line.
- **Tokenization:** Identifies and categorizes lexical elements such as:
 - **Keywords:** number, write, repeat, times, newline, and
 - **Identifiers:** Must start with a letter, can include letters, digits, underscores, and must be ≤ 20 characters
 - **Integer Constants:** No real numbers or + signs allowed
 - **String Literals:** Must be enclosed in double quotes ("")
 - **Operators:** :=, +=, -=
 - **Braces and Semicolons:** {, }, and ; for block structure and line endings
- **Symbol Table Management:** Keeps track of declared variables to detect:
 - Redclarations
 - Use of undeclared variables
- **Error Reporting:** Flags errors such as:
 - Invalid identifier formats
 - Overlong identifiers
 - Invalid or unknown characters
 - Unclosed comments or string literals
 - Improper block nesting

2) CODE DESCRIPTION

2.a) Libraries & Constants

```
C la.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  // Constants
7  #define TOKEN_CAPACITY 256          // Max size of a token
8  #define VARIABLE_LIMIT 100         // Max number of variables
9
```

- `<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<ctype.h>`: Used for file operations, string manipulation, and character analysis.
- `MAX_TOKEN`, `MAX_VARIABLES`: Define limits for token size and number of variables.

2.b) Symbol Table

```
// Symbol table to track declared variables
char varList[VARIABLE_LIMIT][21];
int declaredCount = 0;

// Checks if a variable has already been declared
int isAlreadyDeclared(const char *varName) {
    for (int i = 0; i < declaredCount; i++) {
        if (strcmp(varList[i], varName) == 0)
            return 1;
    }
    return 0;
}

// Adds a variable to the symbol table if not already declared
void addToSymbolTable(const char *varName) {
    if (!isAlreadyDeclared(varName) && declaredCount < VARIABLE_LIMIT) {
        strcpy(varList[declaredCount++], varName);
    }
}
```

This code manages the symbol table for declared variables. It stores variable names in `varList` and tracks the count with `declaredCount`. The `isAlreadyDeclared` function checks if a name is already declared, while `addToSymbolTable` adds new variables if they're not duplicates and within the limit. This helps detect undeclared or repeated variables during analysis.

2.c) Token Classification Functions

```
29
30 // File pointers for input and output
31 FILE *inputFile, *outputFile;
32
33 // Check if a string is a keyword
34 int checkIfKeyword(const char *word) {
35     const char *keywords[] = { "number", "write", "repeat", "times", "newline", "and" };
36     for (int i = 0; i < 6; i++) {
37         if (strcmp(word, keywords[i]) == 0)
38             return 1;
39     }
40     return 0;
41 }
42
43 // Check if a string is a valid operator
44 int checkIfOperator(const char *op) {
45     return (strcmp(op, "!=") == 0 || strcmp(op, "+=") == 0 || strcmp(op, "-=") == 0);
46 }
47
48 // Checks if a character can start an identifier (must be letter)
49 int isValidStartChar(char c) {
50     return isalpha(c);
51 }
52
53 // Checks if a character can be part of an identifier (letter, digit, underscore)
54 int isValidIdentifierChar(char c) {
55     return isalnum(c) || c == '_';
56 }
57
```

checkIfKeyword: Checks if a given word matches one of the 6 reserved keywords (number, write, etc.). Returns 1 if it is a keyword, 0 otherwise.

checkIfOperator: Checks if a given word matches one of the 6 reserved keywords (number, write, etc.). Returns 1 if it is a keyword, 0 otherwise.

isValidStartChar: Returns 1 if the character c is a letter. Used to ensure identifiers start with a letter.

isValidIdentifierChar: Returns 1 if the character c is a letter, digit, or underscore. Used to validate the rest of an identifier.

These functions help classify tokens as keywords, operators, or valid identifiers in your lexical analyzer.

2.d) Error Reporting

```
// Reports an error message with the current line number
void logError(const char *message, int lineNumber) {
    fprintf(stderr, "Error [line %d]: %s\n", lineNumber, message);
}
```

The `logError` function shows error messages with the line number where the problem happened. It takes a message and a line number, and prints them to the screen using `fprintf`. This helps the user know what went wrong and where in the code.

2.e) Main Analysis Function

```
// Main lexical analysis function
void runLexer(const char *inputFileName) {
    char outputFileName[256];
    strcpy(outputFileName, inputFileName);
    char *dot = strrchr(outputFileName, '.');
    if (dot) strcpy(dot, ".lx"); // Replace extension
    else strcat(outputFileName, ".lx");

    // Open input and output files
    inputFile = fopen(inputFileName, "r");
    if (!inputFile) {
        perror("Failed to open the input file");
        exit(1);
    }

    outputFile = fopen(outputFileName, "w");
    if (!outputFile) {
        perror("Failed to create the output file");
        exit(1);
    }

    // Initialization of parsing variables
    char ch;
    char tokenBuffer[TOKEN_CAPACITY];
    int tokenIndex = 0, lineNumber = 1;
    int isExpectingBlock = 0;
    int blockNesting = 0;
    int repeatKeywordLine = -1;
    int afterRepeatKeyword = 0;
    int expectingIdentifier = 0;

    // Character-by-character reading
    while ((ch = fgetc(inputFile)) != EOF) {
        if (ch == '\n') {
            lineNumber++;
            if (isExpectingBlock && !afterRepeatKeyword) {
                char err[100];
                sprintf(err, "Expected '{' after 'repeat ... times' at line %d", repeatKeywordLine);
                logError(err, lineNumber);
            }
        }
    }
}
```

Ln 66 Col 43

```

        isExpectingBlock = 0;
    }
    afterRepeatKeyword = 0;
    continue;
}

if (isspace(ch)) continue;

// If a declaration expects an identifier
if (expectingIdentifier) {
    if (!isalpha(ch)) {
        if (isdigit(ch)) {
            logError("Identifier must begin with a letter", lineNumber);
        } else if (ch == '_') {
            logError("Identifier cannot begin with an underscore", lineNumber);
        } else {
            char err[100];
            sprintf(err, "Invalid identifier start character: '%c'", ch);
            logError(err, lineNumber);
        }
        while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch));
        expectingIdentifier = 0;
        continue;
    }

    tokenIndex = 0;
    tokenBuffer[tokenIndex++] = ch;
    int isValid = 1;

    // Read rest of identifier
    while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch)) {
        if (tokenIndex >= 20) {
            logError("Identifier length cannot exceed 20 characters", lineNumber);
            isValid = 0;
            while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch));

```

```

        break;
    }
    tokenBuffer[tokenIndex++] = ch;
}

tokenBuffer[tokenIndex] = '\0';
ungetc(ch, inputFile);
expectingIdentifier = 0;

if (!isValid) continue;

if (isAlreadyDeclared(tokenBuffer)) {
    char err[100];
    sprintf(err, "Variable '%s' is declared more than once", tokenBuffer);
    logError(err, lineNumber);
    continue;
}

fprintf(outputFile, "Identifier(%s)\n", tokenBuffer);
addToSymbolTable(tokenBuffer);
continue;
}

// Handle comment: * ... *
if (ch == '*') {
    int commentEnded = 0;
    int commentStartLine = lineNumber;
    while ((ch = fgetc(inputFile)) != EOF && ch != '\n') {
        if (ch == '*') {
            commentEnded = 1;
            break;
        }
    }
    if (!commentEnded) {
        char err[100];

```

```

        if (!commentEnded) {
            char err[100];
            sprintf(err, "Unclosed comment started on line %d", commentStartLine);
            logError(err, lineNumber);
        }
        if (ch == '\n') lineNumber++;
        continue;
    }

    // Handle string constant
    if (ch == '"') {
        tokenIndex = 0;
        tokenBuffer[tokenIndex++] = ch;
        int stringClosed = 0;
        while ((ch = fgetc(inputFile)) != EOF) {
            if (ch == '\n') lineNumber++;
            tokenBuffer[tokenIndex++] = ch;
            if (ch == '"') {
                stringClosed = 1;
                break;
            }
        }
        if (!stringClosed) {
            logError("String literal was not closed", lineNumber);
            continue;
        }
        tokenBuffer[tokenIndex] = '\0';
        fprintf(outputFile, "StringConstant(%s)\n", tokenBuffer);
        continue;
    }

    // Handle identifier or keyword
    if (isalpha(ch)) {
        tokenIndex = 0;
        tokenBuffer[tokenIndex++] = ch;
        int isValid = 1;

        while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch)) {
            if (tokenIndex >= 20) {
                logError("Identifier too long", lineNumber);
                isValid = 0;
                while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch));
                break;
            }
            tokenBuffer[tokenIndex++] = ch;
        }

        tokenBuffer[tokenIndex] = '\0';
        ungetc(ch, inputFile);
        if (!isValid) continue;

        if (checkIfKeyword(tokenBuffer)) {
            fprintf(outputFile, "Keyword(%s)\n", tokenBuffer);

            if (strcmp(tokenBuffer, "repeat") == 0) {
                isExpectingBlock = 1;
                repeatKeywordLine = lineNumber;
                afterRepeatKeyword = 1;
            } else if (strcmp(tokenBuffer, "times") == 0) {
                afterRepeatKeyword = 1;
            } else {
                afterRepeatKeyword = 0;
            }

            if (strcmp(tokenBuffer, "number") == 0) {
                expectingIdentifier = 1;
            }
        } else {
            if (!isAlreadyDeclared(tokenBuffer)) {
                char err[100];
                sprintf(err, "Undeclared variable: '%s'", tokenBuffer);
                logError(err, lineNumber);
            }
        }
    }
}

```

```

    }
    fprintf(outputFile, "Identifier(%s)\n", tokenBuffer);
    afterRepeatKeyword = 0;
}
continue;
}

// Identifier starting with underscore
if (ch == '_') {
    logError("Identifiers cannot start with an underscore", lineNumber);
    while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch));
    continue;
}

// Integer constant parsing
if (isdigit(ch) || ch == '-') {
    tokenIndex = 0;
    int isNegative = (ch == '-');
    tokenBuffer[tokenIndex++] = ch;

    if (isNegative) {
        ch = fgetc(inputFile);
        if (isspace(ch)) {
            logError("Negative number must not contain space", lineNumber);
            continue;
        }
        if (ch == '-') {
            logError("Double minus (--number) is invalid", lineNumber);
            continue;
        }
        if (!isdigit(ch)) {
            logError("Invalid format after minus sign", lineNumber);
            continue;
        }
    }
    tokenBuffer[tokenIndex++] = ch;
}

```

```

}

while ((ch = fgetc(inputFile)) != EOF && (isdigit(ch) || ch == '.' || ch == '+' || isalpha(ch) || ch == '_')) {
    if (ch == '.') {
        logError("Real numbers are not supported", lineNumber);
        while ((ch = fgetc(inputFile)) != EOF && (isdigit(ch) || ch == '.'));
        break;
    }
    if (ch == '+') {
        logError("Plus sign in numbers is not valid", lineNumber);
        while ((ch = fgetc(inputFile)) != EOF && isdigit(ch));
        break;
    }
    if (isalpha(ch) || ch == '_') {
        logError("Number cannot contain letters or underscores", lineNumber);
        while ((ch = fgetc(inputFile)) != EOF && isValidIdentifierChar(ch));
        break;
    }
    tokenBuffer[tokenIndex++] = ch;
}

tokenBuffer[tokenIndex] = '\0';
ungetc(ch, inputFile);
fprintf(outputFile, "IntConstant(%s)\n", tokenBuffer);
continue;
}

// Operators
if (ch == ':' || ch == '+' || ch == '-') {
    char next = fgetc(inputFile);
    tokenBuffer[0] = ch;
    tokenBuffer[1] = next;
    tokenBuffer[2] = '\0';
    if (checkIfOperator(tokenBuffer)) {
        fprintf(outputFile, "Operator(%s)\n", tokenBuffer);
    }
}

```

```

        continue;
    } else {
        logError("Invalid operator encountered", lineNumber);
        afterRepeatKeyword = 0;
        continue;
    }
}

// Open/close block
if (ch == '{') {
    fprintf(outputFile, "OpenBlock\n");
    if (isExpectingBlock) {
        isExpectingBlock = 0;
        blockNesting++;
    }
    afterRepeatKeyword = 0;
    continue;
}

if (ch == '}') {
    if (blockNesting > 0) {
        fprintf(outputFile, "CloseBlock\n");
        blockNesting--;
    } else {
        logError("Extra closing block detected", lineNumber);
    }
    afterRepeatKeyword = 0;
    continue;
}

// Statement end
if (ch == ';') {
    fprintf(outputFile, "EndOfLine\n");
    afterRepeatKeyword = 0;
    continue;
}

```

```

        continue;
    }

    // Any other unexpected character
    char err[100];
    sprintf(err, "Unexpected character '%c'", ch);
    logError(err, lineNumber);
    afterRepeatKeyword = 0;
}

if (blockNesting > 0) {
    logError("One or more blocks are not closed", lineNumber);
}

fclose(inputFile);
fclose(outputFile);
printf("Lexical analysis complete. Output saved to %s\n", outputFileName);
}

```

The `runLexer` function is the core of the lexical analyzer. It takes the name of a `.plus` source file as input, constructs a corresponding `.lx` output file name by replacing or appending the `.lx` extension, and opens both files for reading and writing. If either file cannot be opened, the program exits with an error.

At the beginning of the function, several variables are initialized to manage token parsing, track the current line number, and handle language-specific syntax (such as block structures and repeat loops). These include `tokenBuffer` for storing each token, `tokenIndex` to track its length, and flags such as `isExpectingBlock`, `afterRepeatKeyword`, and `expectingIdentifier` to manage parsing context.

The function then enters a loop that reads the input file ****character by character****. For each character, it performs the following checks and actions:

Newline Handling: If a newline is encountered, the line counter is incremented. If a ``repeat ... times`` statement was detected previously but not followed by ``{``, it logs an error.

Whitespace Handling: Whitespace characters are skipped.

Identifier Declarations: If an identifier is expected (after the ``number`` keyword), the function checks that it starts with a letter, is at most 20 characters long, and is not already declared. If valid, the identifier is added to the symbol table.

Comments: Comments must begin and end with ``*`` on the same line. If the end ``*`` is missing before the newline, an "unclosed comment" error is reported.

String Constants: Strings must be enclosed in double quotes (``""``). The function reads characters into the token buffer until another ``""`` is found. If a newline is encountered before closure, an error is reported.

Keywords and Identifiers: If a token starts with a letter, the function reads it fully and checks whether it's a keyword (like ``write``, ``repeat``, etc.) using ``checkIfKeyword()``. If it's not a keyword, it checks whether the variable has been declared, otherwise logs an error. Keywords like ``repeat`` and ``number`` also change the state of the parser by setting flags.

Invalid Identifiers: If an identifier starts with ``_``, or is too long, an error is reported and the rest of the token is skipped.

Integer Constants: Numbers must be valid integers. The function handles optional minus signs, disallows spaces between ``-`` and the number, and rejects real numbers (containing ``.``), numbers with a ``+``, or mixed characters like ``12abc``.

Operators: If the character is ``:``, ``+``, or ``-``, the function looks ahead to form a two-character operator and checks if it's valid using ``checkIfOperator()``. Invalid combinations are flagged.

Braces and Semicolons: ``{`` and ``}`` are used to track code blocks, and ``;`` indicates the end of a statement. The function counts opening and closing braces to ensure blocks are properly matched.

Unexpected Characters: Any unrecognized character triggers a generic error message.

Finally, after reading the file, if the number of opened blocks does not match the number of closed blocks, an error about unclosed blocks is reported. The input and output files are then closed, and a success message is printed.

2.f) Entry Point

```
// Main function: handles command-line arguments and triggers processing
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <sourcefile.plus>\n", argv[0]);
        return 1;
    }

    char fileName[256];
    snprintf(fileName, sizeof(fileName), "%s.plus", argv[1]);
    runLexer(fileName);

    return 0;
}
```

This is the main function that starts your lexical analyzer. It checks if the user gave a file name when running the program. If not, it shows a usage message and exits. If a file name is given, it adds ".plus" to it, then calls the `runLexer` function to analyze that file. At the end, it returns 0 to show the program finished successfully.

3) ERROR HANDLING

3.a) Invalid or Incorrect Identifiers

Input:

```
≡ myscript.plus
1  number 1size;
2  number _sum;
3  number thisIdentifierNameIsDefinitelyTooLong;
4  number valid;
5  number valid;
6  write undeclaredVar;
7  
```

Output:

```
● ceylintor@192 Source Code % gcc la.c -o la
● ceylintor@192 Source Code % ./la myscript
Error [line 1]: Identifier must begin with a letter
Error [line 2]: Identifier cannot begin with an underscore
Error [line 3]: Identifier length cannot exceed 20 characters
Error [line 5]: Variable 'valid' is declared more than once
Error [line 6]: Undeclared variable: 'undeclaredVar'
Lexical analysis complete. Output saved to myscript.lx
○ ceylintor@192 Source Code %
```

≡ myscript.lx

```
1 Keyword(number)
2 Keyword(number)
3 Keyword(number)
4 EndOfLine
5 Keyword(number)
6 Identifier(valid)
7 EndOfLine
8 Keyword(number)
9 EndOfLine
10 Keyword(write)
11 Identifier(undeclaredVar)
12 EndOfLine
13
```

3.b) Unclosed Comment

Input:

≡ myscript.plus

```
1 number size;
2 * this comment is not closed
3 write size;
4
```

Output:

```
● ceylintor@192 Source Code % gcc la.c -o la
● ceylintor@192 Source Code % ./la myscript
Error [line 2]: Unclosed comment started on line 2
Lexical analysis complete. Output saved to myscript.lx
○ ceylintor@192 Source Code %
```

```
≡ myscript.lx
1 Keyword(number)
2 Identifier(size)
3 EndOfLine
4 Keyword(write)
5 Identifier(size)
6 EndOfLine
7
```

3.c) Unclosed String

Input:

```
≡ myscript.plus
1 number size;
2 write "This string is not closed;
3
```

Output:

```
● ceylintor@192 Source Code % gcc la.c -o la
● ceylintor@192 Source Code % ./la myscript
Error [line 3]: String literal was not closed
Lexical analysis complete. Output saved to myscript.lx
○ ceylintor@192 Source Code %
```

```
≡ myscript.lx
1 Keyword(number)
2 Identifier(size)
3 EndOfLine
4 Keyword(write)
5
```

3.d) Invalid Numbers

Input:

```
≡ myscript.plus
1  number a;
2  a := 3.14;
3  a := +5;
4  a := --2;
5  a := 12abc;
6
```

Output:

```
● ceylinton@192 Source Code % gcc la.c -o la
● ceylinton@192 Source Code % ./la myscript
Error [line 2]: Real numbers are not supported
Error [line 3]: Invalid operator encountered
Error [line 4]: Double minus (--number) is invalid
Error [line 5]: Number cannot contain letters or underscores
Lexical analysis complete. Output saved to myscript.lx
○ ceylinton@192 Source Code %
```

```
≡ myscript.lx
1  Keyword(number)
2  Identifier(a)
3  EndOfLine
4  Identifier(a)
5  Operator(:=)
6  IntConstant(3)
7  EndOfLine
8  Identifier(a)
9  Operator(:=)
10 EndOfLine
11 Identifier(a)
12 Operator(:=)
13 IntConstant(2)
14 EndOfLine
15 Identifier(a)
16 Operator(:=)
17 IntConstant(12)
18 EndOfLine
19
```

3.e) Missing "{" after repeat block

Input:

```
≡ myscript.plus
1  number size;
2  size := 3;
3  repeat size times
4  write size;
5  
```

Output:

```
● ceylintor@192 Source Code % gcc la.c -o la
● ceylintor@192 Source Code % ./la myscript
Error [line 5]: Expected '{' after 'repeat ... times' at line 3
Lexical analysis complete. Output saved to myscript.lx
○ ceylintor@192 Source Code % 
```

```
≡ myscript.lx
1  Keyword(number)
2  Identifier(size)
3  EndOfLine
4  Identifier(size)
5  Operator(:=)
6  IntConstant(3)
7  EndOfLine
8  Keyword(repeat)
9  Identifier(size)
10 Keyword(times)
11 Keyword(write)
12 Identifier(size)
13 EndOfLine
14 
```

3.f) More "}"

Input:

```
≡ myscript.plus
1  number x;
2  {
3  | write x;
4  | }
5  number y;
6  
```

Output:

```
● ceylintor@192 Source Code % gcc la.c -o la
● ceylintor@192 Source Code % ./la myscript
Error [line 4]: Extra closing block detected
Error [line 4]: Extra closing block detected
Lexical analysis complete. Output saved to myscript.lx
○ ceylintor@192 Source Code %
```

```
≡ myscript.lx
1  Keyword(number)
2  Identifier(x)
3  EndOfLine
4  OpenBlock
5  Keyword(write)
6  Identifier(x)
7  EndOfLine
8  Keyword(number)
9  Identifier(y)
10 EndOfLine
```

4) GIVEN SCRIPT (NO ERROR)

Input:

```
≡ myscript.plus
1  number size;
2  number sum;
3  size:=5;
4  repeat size times      *ignore me, I am a comment*
5  { write size and newline; *print to screen*
6  |   sum+=size;
7  }
8  write newline and "Sum:" and sum;
```

Output:

```
● ceylintor@192 Source Code % gcc la.c -o la
● ceylintor@192 Source Code % ./la myscript
Lexical analysis complete. Output saved to myscript.lx
○ ceylintor@192 Source Code %
```

≡ myscript.lx

```
1 Keyword(number)
2 Identifier(size)
3 EndOfLine
4 Keyword(number)
5 Identifier(sum)
6 EndOfLine
7 Identifier(size)
8 Operator(:=)
9 IntConstant(5)
10 EndOfLine
11 Keyword(repeat)
12 Identifier(size)
13 Keyword(times)
14 OpenBlock
15 Keyword(write)
16 Identifier(size)
17 Keyword(and)
18 Keyword(newline)
19 EndOfLine
20 Identifier(sum)
21 Operator(+=)
22 Identifier(size)
23 EndOfLine
24 CloseBlock
25 Keyword(write)
26 Keyword(newline)
27 Keyword(and)
28 StringConstant("Sum:")
29 Keyword(and)
30 Identifier(sum)
31 EndOfLine
32
```